

H&M Personalized Fashion Recommendations



Provide product recommendations based on previous purchases

[H&M Personalized Fashion Recommendations | Kaggle](#)

比赛介绍

在这次竞赛中，参赛者将根据H&M集团以往交易的数据以及客户和商品的元数据来开发产品推荐。可用的元数据从简单的数据，如服装类型和客户年龄，到产品描述的文本数据，再到服装图片的图像数据。

对于哪些信息可能是有用的，没有任何先入为主的想法--这要由你自己去发现。如果你想研究分类数据类型的算法，或深入研究NLP和图像处理的深度学习，那是由你决定的。

- 竞赛类型：本次竞赛属于推荐系统/多模态，这边推荐使用的模型：LightGBM/Catboost/CNN/Bert 等
- 赛题数据：数据集包含交易数据、客户元数据和商品元数据（包含商品描述文本、商品图片）。
- 评估标准：**Mean Average Precision @ 12 (MAP@12)** [Understanding Mean Average Precision | Kaggle](#)
- 推荐阅读 Kaggle内的Tutorial EDA来获取一些更详细的预备知识：[H&M EDA FIRST LOOK | Kaggle](#)
- 本次比赛LGBM Baseline 推荐：[Radek's LGBMRanker starter-pack | Kaggle](#)

数据说明

官方数据页面：[H&M Personalized Fashion Recommendations | Kaggle](#)

数据集包含交易数据、客户元数据和商品元数据（包含商品描述文本、商品图片）。

- **images/** - 每个article_id对应的图片文件夹；注意，不是所有的article_id值都有对应的图片。
- **articles.csv** - 每个商品的article_id的详细元数据
- **customers.csv** - 每个用户的customer_id的元数据
- **sample_submission.csv** - 正确格式的提交样本文件
- **transactions_train.csv** - 训练数据，包括每个客户在每个日期的购买情况，以及其他信息。重复的行对应于同一物品的多次购买。你的任务是预测每个顾客在紧接训练数据期之后的7天内将购买的物品_ID。

注意：你必须对提交的样本中发现的所有customer_id值进行预测。所有在测试期间进行购买的客户都会被打分，无论他们在训练数据中是否有购买记录。

解决方案思路

本次竞赛我们机器配置如下：

GPU: 3090

CPU: i9-10900k

RAM: 128G

OS: Ubuntu18.04

方案策略如下：

- 本次竞赛我们只采用了数据集中的表格数据部分，并没有使用图像数据和文本数据，我们最终使用的模型是Catboost。
- 首先，因为pandas读取csv文件数据较慢，所以我们在一开始就将所有表格数据转为了pickle格式保存，并在之后直接读取pkl文件，这让数据加载的时间缩短了10倍以上。
- 特征工程部分是本次比赛的重点，我们主要通过以下四个步骤构建新的特征：
 1. 之后我们创建了user-item矩阵，并使用LightFM库，训练了user的embedding，作为后续模型输入的特征。
 2. 我们对各个商品的属性特征做onehot编码，接着并入交易表，然后groupby users聚合。构建了user和商品属性之间的关系特征。
 3. 我们通过上述新特征，生成了candidates候选项，并且衍生出少量rank特征。
 4. 正如讲义2中提到，我们还生成了user的静态特征，user动态特征，item静态特征，item动态特征，user-item对静态特征，user-item对动态特征等等。
- 之后数据集切分上，我们首先将最后一周作为验证集，其余周做为训练集，使用Catboost进行训练，CV MAP 0.324。

比赛上分历程

1. 使用LightGBM Baseline训练模型。少量特征 + 粗糙的 candidates挑选，PB：0.0203
2. 加入并调参 LightFM 生成user的embedding，PB：0.0231
3. 加入商品属性特征onehot编码，并用user聚合，PB：0.0264
4. 增加了repurchase等多个candidates条件，PB：0.0271
5. 增加了user-item对的静态特征和动态特征，PB：0.0285
6. 使用Catboost 并调参，PB：0.0309
7. 增加了user、item、user-item对新鲜度特征，PB：0.0315+

特征生成

```
# user静态特征，加入年龄
df = df.merge(users[['user', 'age']], on='user')
```

```

# item静态特征, 所有idx结尾特征
item_features = [c for c in items.columns if c.endswith('idx')]
df = df.merge(items[['item'] + item_features], on='item')

# user动态特征 (transactions)
week_end = week + CFG.user_transaction_feature_weeks
# 交易表中, 每个用户在某个时间段的[price 和 sales_channel_id]的[平均值, 标准差]
tmp = transactions.query("@week <= week < @week_end").groupby('user')[['price',
'sales_channel_id']].agg(['mean', 'std'])
tmp.columns = ['user_' + '_' .join(a) for a in tmp.columns.to_flat_index()] # 加入列名后缀
, 如 user_price_mean, user_price_std, user_sales_channel_id_mean,
user_sales_channel_id_std
df = df.merge(tmp, on='user', how='left') # merge新列

# item动态特征 (transactions)
week_end = week + CFG.item_transaction_feature_weeks
# 交易表中, 每个item在某个时间段的[price 和 sales_channel_id]的[平均值, 标准差]
tmp = transactions.query("@week <= week < @week_end").groupby('item')[['price',
'sales_channel_id']].agg(['mean', 'std'])
tmp.columns = ['item_' + '_' .join(a) for a in tmp.columns.to_flat_index()] # 加入列名后缀
df = df.merge(tmp, on='item', how='left') # merge新列

# item动态特征 (user features)
week_end = week + CFG.item_age_feature_weeks
# 交易表某段week, 加入年龄列
tmp = transactions.query("@week <= week < @week_end").merge(users[['user', 'age']],
on='user')
tmp = tmp.groupby('item')['age'].agg(['mean', 'std']) # 每个item的购买年龄的[mean和std]
tmp.columns = [f'age_{a}' for a in tmp.columns.to_flat_index()] # 加入列名后缀
df = df.merge(tmp, on='item', how='left') # merge新列

# item新鲜度特征
tmp = transactions.query("@week <= week").groupby('item')
['day'].min().reset_index(name='item_day_min') # 指定week及之前, 每个item的最近一次被购买的
day
tmp['item_day_min'] -= transactions.query("@week == week")['day'].min() # item有多少天没
有被购买了。
df = df.merge(tmp, on='item', how='left') # merge新列

# 每个item被购买量
week_end = week + CFG.item_volume_feature_weeks
tmp = transactions.query("@week <= week <
@week_end").groupby('item').size().reset_index(name='item_volume') # 时间范围内每个item的
被购买量
df = df.merge(tmp, on='item', how='left')

# user新鲜度特征

```

```

tmp = transactions.query("@week <= week").groupby('user')
['day'].min().reset_index(name='user_day_min') # 每个user最近一次购买的day
tmp['user_day_min'] -= transactions.query("@week == week")['day'].min() # 该user有多少天
没有购买了
df = df.merge(tmp, on='user', how='left')

# 每个user购买量
week_end = week + CFG.user_volume_feature_weeks
tmp = transactions.query("@week <= week <
@week_end").groupby('user').size().reset_index(name='user_volume') # 每个user购买量
df = df.merge(tmp, on='user', how='left')

# user-item对新鲜度特征
tmp = transactions.query("@week <= week").groupby(['user', 'item'])
['day'].min().reset_index(name='user_item_day_min') # 每个user最近一次购买某个item的day
tmp['user_item_day_min'] -= transactions.query("@week == week")['day'].min() # 该user有
多少天没有购买该item了
df = df.merge(tmp, on=['item', 'user'], how='left')

# user-item购买量
week_end = week + CFG.user_item_volume_feature_weeks
tmp = transactions.query("@week <= week < @week_end").groupby(['user',
'item']).size().reset_index(name='user_item_volume')
df = df.merge(tmp, on=['user', 'item'], how='left')

# lfm features
seen_users = transactions.query("week >= @pretrain_week")['user'].unique() # lfm_week 之
前的 user (或者称为已知用户)
user_reps = calc_embeddings('i_i', pretrain_week, 16) # 获取user_embeddings
user_reps = user_reps.query("user in @seen_users") # user 属于已知用户
df = df.merge(user_reps, on='user', how='left')

```

Model

```

train_dataset = catboost.Pool(data=train[feature_columns], label=train['y'],
group_id=train['query_group'], cat_features=cat_features)
valid_dataset = catboost.Pool(data=valid[feature_columns], label=valid['y'],
group_id=valid['query_group'], cat_features=cat_features)

params = {
    'loss_function': 'YetiRank',
    'use_best_model': True,
    'one_hot_max_size': 300,
    'iterations': 10000,
}

model = catboost.CatBoost(params) # 创建模型

```

```

model.fit(train_dataset, eval_set=valid_dataset) # 训练模型

plt.plot(model.get_evals_result()['validation']['PFound']) # 打印验证结果

# 特征重要度
feature_importance = model.get_feature_importance(train_dataset)
sorted_idx = np.argsort(feature_importance)
plt.figure(figsize=(8, 16))
plt.yticks(range(len(feature_columns)), np.array(feature_columns)[sorted_idx])
plt.barh(range(len(feature_columns)), feature_importance[sorted_idx])

```

MAP@12 代码

```

#
https://github.com/benhamner/Metrics/blob/master/Python/ml\_metrics/average\_precision.py

def apk(actual, predicted, k=12):
    """
    Computes the average precision at k.

    This function computes the average prescision at k between two lists of
    items.

    Parameters
    -----
    actual : list
        A list of elements that are to be predicted (order doesn't matter)
    predicted : list
        A list of predicted elements (order does matter)
    k : int, optional
        The maximum number of predicted elements

    Returns
    -----
    score : double
        The average precision at k over the input lists

    """
    if len(predicted) > k:
        predicted = predicted[:k] # 截断12个预测值

    score = 0.0
    num_hits = 0.0

    for i, p in enumerate(predicted):
        if p in actual and p not in predicted[:i]: # p not in predicted[:i] 意味着 pred不
            能重复出现

```

```

        num_hits += 1.0
        score += num_hits / (i + 1.0)

    if not actual:
        return 0.0 # gt为空, 返回0

    return score / min(len(actual), k)

def mapk(actual, predicted, k=12):
    """
    Computes the mean average precision at k.

    This function computes the mean average prescision at k between two lists
    of lists of items.

    Parameters
    -----
    actual : list
        A list of lists of elements that are to be predicted
        (order doesn't matter in the lists)
    predicted : list
        A list of lists of predicted elements
        (order matters in the lists)
    k : int, optional
        The maximum number of predicted elements

    Returns
    -----
    score : double
        The mean average precision at k over the input lists

    """
    return np.mean([apk(a, p, k) for a, p in zip(actual, predicted)])

print('mAP@12:', mapk(merged['gt'], merged['item'])) # 计算验证集score

```

模型调参

```

# lgbm参数 https://lightgbm.apachecn.org/#/docs/6
params = {
    'objective': 'regression', # 回归问题
    'boosting_type': 'gbdt', # 提升树
    'n_jobs': -1, # cpu线程数
    'verbose': -1, # 显示日志
    'seed': SEED,

```

```

'feature_fraction_seed': SEED,
'bagging_seed': SEED,
'drop_seed': SEED,
'data_random_seed': SEED,

'max_bin': trial.suggest_int("max_bin", 80, 255), # 最大分箱数量
'learning_rate': trial.suggest_loguniform("learning_rate", 0.003, 0.4), # 学习率
'num_leaves': trial.suggest_int("num_leaves", 10, 400), # 最大叶子数量
'max_depth': trial.suggest_int("max_depth", 3, 64), # 最大树深度
# 一个叶子上数据的最小数量
'min_child_samples': trial.suggest_int("min_child_samples", 16, 600),
# 一个叶子上的最小 hessian 和
'min_child_weight': trial.suggest_uniform("min_child_weight", 7e-4, 2e-2),
# LGBM 将会在每次迭代中随机选择部分特征
'feature_fraction': trial.suggest_discrete_uniform("feature_fraction", 0.05, 0.8,
0.1),
# LGBM将在每个树节点上随机选择一个特征子集
'feature_fraction_bynode': trial.suggest_discrete_uniform("feature_fraction_bynode",
0.2, 0.9, 0.1),
# 不进行重采样的情况下随机选择部分数据
'bagging_fraction': trial.suggest_discrete_uniform("bagging_fraction", 0.2, 1.0,
0.1),
'bagging_freq': trial.suggest_int('bagging_freq', 10, 100), # 每 k 次迭代执行bagging

'reg_alpha': trial.suggest_categorical("reg_alpha", [0, 0.001, 0.01, 0.03, 0.08,
0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]), # 11 正则化
'reg_lambda': trial.suggest_categorical("reg_lambda", [0, 0.001, 0.01, 0.03, 0.08,
0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]), # 12 正则化
}

```

代码、数据集

- 代码
 - HM_get_data.ipynb # 生成pickle特征
 - HM_feature.ipynb # 特征工程
 - HM_train_infer # 训练和推理
- 数据集
 - 官网数据: [H&M Personalized Fashion Recommendations | Kaggle](#)

TL;DR

竞赛是由H&M举办的服装方向的多模态推荐系统大赛，参赛者可以使用以往交易的数据以及客户和商品的元数据来预测未来7天的购买商品情况。本次竞赛中我们团队选择了纯表格数据，采用了多种Recall方法来做candidate generation，并且计算了用户向量和物品向量之间的相似度作为重要的特征。此外我们还创建了用户和物品的静态特征和动态特征，并使用Catboost模型来训练所有特征数据，最终我们取得Private Leaderboard银牌成绩。

